Cheat sheet: NAMD and OpenMM files

Keep this open while you read `nve.namd` and `nve.py` in class.

Every MD run needs four things

Ingredient	What it is	In this tutorial
Topology + force field	atoms, bonds, charges, Lennard-Jones parameters	`inputs/trpcage_wat.prmtop`
Coordinates	x, y, z of every atom, plus the box size	`inputs/trpcage_equil.rst7`
Settings	cutoff, PME, timestep, ensemble, run length	`namd/nve.namd` or `openmm/nve.py`
Output	energies, trajectory, restart files	NAMD: `.log .dcd .coor .vel .xsc` · OpenMM: `.csv .dcd`

Both engines read the **same** `.prmtop` and `.rst7`, so their results can be compared directly. (How these two files were made is in `inputs/system_prep/`. You do not need it.)

NAMD

You run it from the terminal: `namd2 +p4 nve.namd > out/nve.log` (`+p4` = use 4 CPU cores).

`nve.namd` is a **config file**: one `keyword value` per line, no program logic. The keywords to know:

Keyword	Meaning
`amber yes`, `parmfile`, `ambercoor`	read Amber-format topology and coordinates
`cutoff 9.0`, `PME yes`	nonbonded forces: 9 Å cutoff, long-range electrostatics with PME
`timestep 2.0`	2 fs per step
`rigidBonds all`	bonds to hydrogen are fixed in length, which is what makes 2 fs possible
`cellBasisVector1/2/3`, `cellOrigin`	the periodic box (NAMD does not read it from the prmtop)
`temperature 300`	give atoms random starting velocities for 300 K
`minimize N` / `run N`	N steps of energy minimisation / N steps of dynamics
`outputName out/nve`	prefix for every output file
`outputEnergies 100`	print energies every 100 steps

The ensemble is chosen by which keywords are present. `langevin on` adds a thermostat (NVT), `langevinPiston on` adds a barostat (NPT). **Neither present = NVE.**

NAMD output (all start with the `outputName` prefix)

File	Contents	Used for
`nve.log`	everything printed, including the `ENERGY:` lines	energy plots
`nve.dcd`	trajectory (binary)	watching in VMD
`nve.coor` / `nve.vel` / `nve.xsc`	final positions / velocities / box	continuing the run later

Reading the log: each `ENERGY:` line is one snapshot. The `ETITLE:` line above them names the columns: `TS BOND ANGLE DIHED IMPRP ELECT VDW BOUNDARY MISC KINETIC TOTAL TEMP POTENTIAL ...`. Energies are in **kcal/mol**. For NVE we look at **KINETIC, POTENTIAL, TOTAL** and **TEMP**.

```
grep ^ETITLE out/nve.log | head -1     # column names
grep ^ENERGY out/nve.log | tail -3     # latest values
```

OpenMM

OpenMM is a **Python library**: you write a short script, and each block matches a part of the NAMD file.

```
prmtop = app.AmberPrmtopFile('trpcage_wat.prmtop')        # topology       (= parmfile)
inpcrd = app.AmberInpcrdFile('trpcage_equil.rst7')        # coords + box   (= ambercoor + cellBasisVector)
system = prmtop.createSystem(
    nonbondedMethod=app.PME,                              # (= PME yes)
    nonbondedCutoff=0.9*unit.nanometer,                   # (= cutoff 9.0)
    constraints=app.HBonds)                               # (= rigidBonds)
integrator = mm.VerletIntegrator(0.002*unit.picoseconds)  # 2 fs, NVE    (= timestep, no langevin)
sim = app.Simulation(prmtop.topology, system, integrator)
sim.context.setPositions(inpcrd.positions)
sim.context.setVelocitiesToTemperature(300*unit.kelvin)   # (= temperature 300)
sim.step(1000)                                            # (= run 1000)
```

- **The ensemble is chosen by the integrator and forces.** `VerletIntegrator` with no barostat = NVE. `LangevinMiddleIntegrator` = NVT. Adding a `MonteCarloBarostat` = NPT.
- **Units differ from NAMD:** nm, ps, kJ/mol (NAMD: Å, fs, kcal/mol). 1 kcal = 4.184 kJ. `plot_nve.py` converts everything to kcal/mol.
- Output comes from **reporters** attached to the simulation: `StateDataReporter` writes `nve.csv` (energies), `DCDReporter` writes `nve.dcd` (trajectory).
- `nve.csv` columns: step, time (ps), potential, kinetic, total energy (kJ/mol), temperature (K).

Side by side

Concept	NAMD	OpenMM
Load force field	`amber yes`, `parmfile`	`AmberPrmtopFile`
Cutoff / PME	`cutoff 9.0`, `PME yes`	`nonbondedMethod=PME`, `nonbondedCutoff`
Rigid X-H bonds	`rigidBonds all`	`constraints=HBonds`
Timestep	`timestep 2.0` (fs)	`VerletIntegrator(0.002*unit.picoseconds)`
Starting velocities	`temperature 300`	`setVelocitiesToTemperature(300*kelvin)`
NVE	no `langevin`, no `langevinPiston`	`VerletIntegrator`, no barostat
NVT	`langevin on`	`LangevinMiddleIntegrator`
NPT	`langevin on` + `langevinPiston on`	Langevin + `MonteCarloBarostat`
Minimise	`minimize 2000`	`sim.minimizeEnergy()`
Run	`run 5000`	`sim.step(5000)`
Energies	`ENERGY:` lines in the log	`StateDataReporter` → CSV
Units	Å, fs, kcal/mol	nm, ps, kJ/mol